Programmation Orientée Objet (POO) à destination de la robotique

Travaux Pratique : Packaging et publication d'un projet Python

Enseignant : Adam GOUGUET
Mail : adam.gouguet@imt-nord-europe.fr
Année : 2025-2026

Table des matières

1 Objectifs du TP **1**

2 Contexte **2**

3 Organisation du projet **2**

4 Le package Python **2**

5 Point d'entrée du programme **3**

6 Le fichier `pyproject.toml` **3**

7 Installation et vérification **3**
7.1 Installation locale 3
7.2 Vérification des imports 4

8 Exécution du projet **4**

1 Objectifs du TP

À l'issue de ce TP, vous serez capables de :
— structurer un projet Python de manière professionnelle,
— rendre un projet installable via l'outil `pip`,
— définir les dépendances et les métadonnées d'un package,
— comprendre les bases de la distribution logicielle en Python,
— publier un projet comme s'il s'agissait d'un produit logiciel.

Ce TP vise à rapprocher le travail réalisé en cours de pratiques professionnelles courantes en ingénierie logicielle et en robotique.

2 Contexte

Jusqu'à présent, le projet de simulation de robot mobile a été exécuté directement depuis les fichiers sources. Dans un contexte professionnel, un tel projet doit pouvoir être :

— installé facilement sur une autre machine,
— utilisé comme une bibliothèque Python,
— intégré dans d'autres projets.

L'objectif de ce TP est de transformer le projet existant en un **package Python installable**, comme s'il devait être distribué à des utilisateurs finaux ou à des clients.

3 Organisation du projet

Commencez par réorganiser votre dépôt GitHub pour respecter l'arborescence suivante :

```
robot-mobile
├── README.md
├── LICENSE
├── pyproject.toml
└── src
    └── robot
        ├── __init__.py
        ├── __main__.py
        ├── robot.py
        ├── moteur.py
        ├── controleur.py
        └── ...
```

Remarque importante Dans ce TP, nous utilisons une organisation dite `src/`. Cela signifie que le code Python du projet n'est **pas** placé directement à la racine du dépôt, mais dans un sous-dossier dédié. Cette pratique est très courante dans les projets professionnels Python, car elle évite de nombreux problèmes d'imports et force à installer le package correctement avant de l'utiliser.

4 Le package Python

Le dossier `src/robot` correspond à votre **package Python**. C'est ce package qui sera installé sur le système lors de l'exécution de la commande `pip install`.

À faire

— Vérifiez que le dossier `robot` contient bien un fichier `__init__.py`.
— Vérifiez que tous les fichiers Python du projet se trouvent bien dans ce dossier.

Sans le fichier `__init__.py`, Python ne reconnaîtra pas le dossier comme un package.

5 Point d'entrée du programme

Dans un projet Python installable, il est courant de proposer un moyen simple de lancer le programme principal.

Pour cela, Python permet d'utiliser un fichier spécial nommé `__main__.py`.

Principe Si un package contient un fichier `__main__.py`, alors la commande suivante devient possible :

```
python -m robot
```

À faire Dans le fichier `src/robot/__main__.py` :

— écrivez le code permettant de lancer une simulation simple,
— ce fichier doit utiliser les classes définies dans votre package,
— aucun code de simulation ne doit se trouver ailleurs.

Ce fichier joue le rôle de **script principal** de votre projet.

6 Le fichier `pyproject.toml`

Le fichier `pyproject.toml` décrit comment votre projet doit être construit, installé et utilisé par Python. Il doit être placé à la racine du dépôt. Je vous laisse chercher sur internet ce qu'il doit contenir.

> 💡 Petit indice quand même : Doc python pyproject.toml ;)

Vous devez être capables d'installer votre projet avec la commande dans la racine du projet :

```
pip install .
```

7 Installation et vérification

Une fois le projet structuré et configuré, vous devez tester son installation.

7.1 Installation locale

Depuis la racine du projet, exécutez :

```
pip install .
```

7.2 Vérification des imports

Ouvrez un interpréteur Python et vérifiez que vous pouvez importer vos classes depuis n'importe quel dossier :

```
python3
from robot.robot import Robot
from robot.controleur import Controleur
...
```

Si ces imports fonctionnent, alors votre package est correctement installé.

8 Exécution du projet

Une fois le package installé, vous devez être capables de lancer la simulation sans accéder aux fichiers sources.

À faire Testez une des commandes suivantes :

```
python -m robot
```

ou

```
robot-mobile-sim
```

> Les commandes au-dessus dépendant de vos noms de packages et point d'entrée que vous avez définie.

Ces deux commandes doivent lancer la simulation définie dans le fichier `__main__.py`.